



VIT BHOPAL UNIVERSITY

School of Computing Science and Engineering

Bhopal-Indore Highway, Kothrikalan, Sehore

Madhya Pradesh - 466114

**CSA1021 INTRODUCTION TO PROBLEM SOLVING
AND PROGRAMMING**

REG.NO: 26BAI11071

NAME : Nishtha Jain

BRANCH: AIML

SEMESTER: Fall Semester 2026-27

Absolutely. Based on your Rock Paper Scissors Python console game, your project documentation can be prepared in exactly this structure.

Since your project uses in-memory Python dictionaries rather than a database, the database section should be adapted to describe the application's storage/data structure, rather than pretending that an actual database exists.

ROCK PAPER SCISSORS — PROJECT DESIGN DOCUMENTATION

1. Problem Statement

The project aims to develop a console-based Rock Paper Scissors game in Python that allows users to register and log in using their phone number and password before playing.

The system provides multiple subscription plans—Free, Premium, and Pro—with different daily match limits. Users can play Rock Paper Scissors against the computer, view their game statistics, manage their account, and upgrade their subscription through a simulated payment system.

The application uses in-memory storage, meaning user information, subscription details, scores, and payment records are maintained while the program is running and are lost when the application terminates.

2. Objectives

The main objectives of the project are:

- 1. To develop an interactive console-based Rock Paper Scissors**

- game.
2. To provide a user registration and login system.
 3. To securely store passwords using SHA-256 hashing.
 4. To allow users to play Rock Paper Scissors against a computer opponent.
 5. To implement a Best-of-3 match system.
 6. To track user wins and losses.
 7. To implement different subscription plans with daily match limits.
 8. To provide a simulated payment mechanism for Premium and Pro plans.
 9. To allow users to view and manage their account information.
 10. To demonstrate modular programming using Python functions.
 11. To maintain user and game information using an in-memory dictionary.
-

3. Functional Requirements

The functional requirements describe what the system must do.

FR1 — User Registration

- The system shall allow a new user to register.
- The user shall provide:
 - Full name
 - 10-digit phone number
 - Password
 - Password confirmation.
- The system shall prevent duplicate phone numbers.
- The password must contain at least 6 characters and one digit.

FR2 — User Login

- The system shall allow registered users to log in using their phone number and password.
- The system shall allow a maximum of 3 unsuccessful login attempts.
- The password shall be compared using its hashed value.

FR3 — Game Selection

- The user shall be able to choose:
 - Rock
 - Paper
 - Scissors
- The system shall also accept:
 - r for Rock
 - p for Paper
 - s for Scissors.

FR4 — Computer Selection

- The computer shall randomly select Rock, Paper, or Scissors.

FR5 — Winner Determination

The system shall determine the winner according to:

Player	Beats
Rock	Scissors
Scissors	Paper
Paper	Rock

FR6 — Best-of-3 Match

- Each match shall continue until either the player or computer wins 2 rounds.
- The system shall display the result of every round.
- The system shall display the final match result.

FR7 — Score Tracking

The system shall maintain:

- Number of wins
- Number of losses
- Number of draws

FR8 —

Subscription Management

The system shall provide:

Plan	Price	Daily Limit
Free	₹0	5 matches
Premium	₹49	20 matches
Pro	₹99	1000 matches*

*The code implements Pro as 1000 matches/day, although the user-facing text describes it as "Unlimited."

FR9 — Payment

- The system shall simulate payment for paid plans.
- The user shall enter:
 - Card number
 - Expiry date
 - CVV.
- The system shall validate the input format.
- Successful payments shall be recorded in memory.

FR10 — Daily Match Limit

- The system shall track the number of matches played on the current date.
- The system shall reset the daily match count when a new date is detected.
- The system shall prevent additional matches after reaching the plan limit.

FR11 — Profile Management

The user shall be able to:

- View name
- View phone number

- View subscription
- View wins/losses/draws
- Change their password.

FR12 — Logout

- The user shall be able to log out and return to the main menu.
-

4.Non-functional Requirements

NFR1 — Usability

The application should provide simple, text-based menus that are easy to understand.

NFR2 — Security

Passwords should not be stored as plain text. The system uses SHA-256 hashing.

NFR3 — Performance

Game operations should execute quickly without noticeable delay, apart from the simulated payment-processing delay.

NFR4 — Reliability

The system should validate invalid inputs and prevent the program from crashing during normal user interaction.

NFR5 — Maintainability

The program should be divided into separate functions for registration, login, payment, game logic, subscription management, and profile management.

NFR6 — Portability

The program should run on systems supporting Python.

NFR7 — Data Persistence

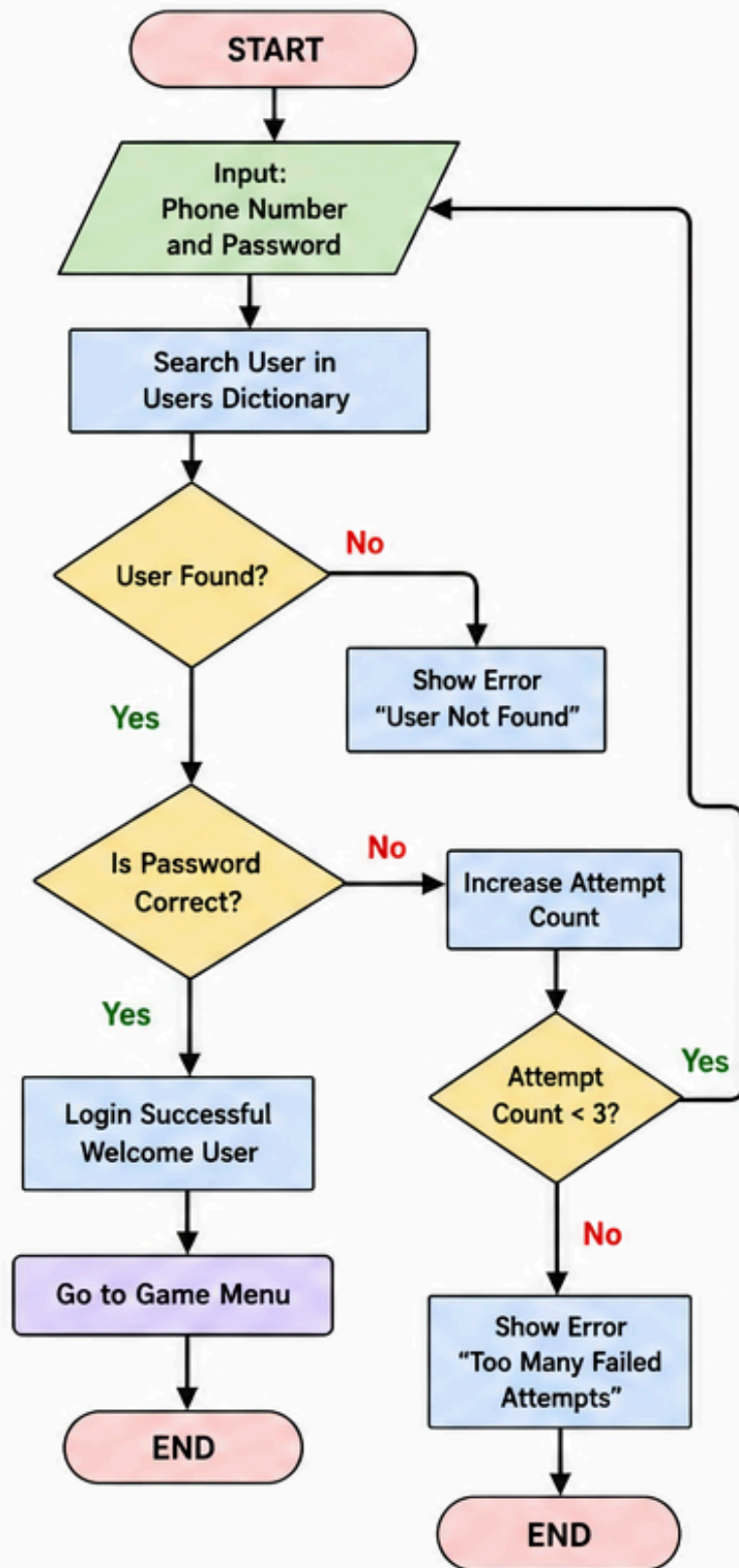
The current implementation does not provide permanent data storage. All information is stored in memory and is lost when the program terminates.

5.System

Architecture Diagram

The application follows a simple modular console application architecture.

2. LOGIN FLOWCHART



Main Components

User Interface

- Displays menus
- Accepts user input
- Displays results

Authentication Module

- Registration
- Login
- Password hashing

Game Module

- Player choice
- Computer choice
- Winner calculation
- Score tracking

Subscription Module

- Plan selection
- Daily limits
- Payment processing

Account Module

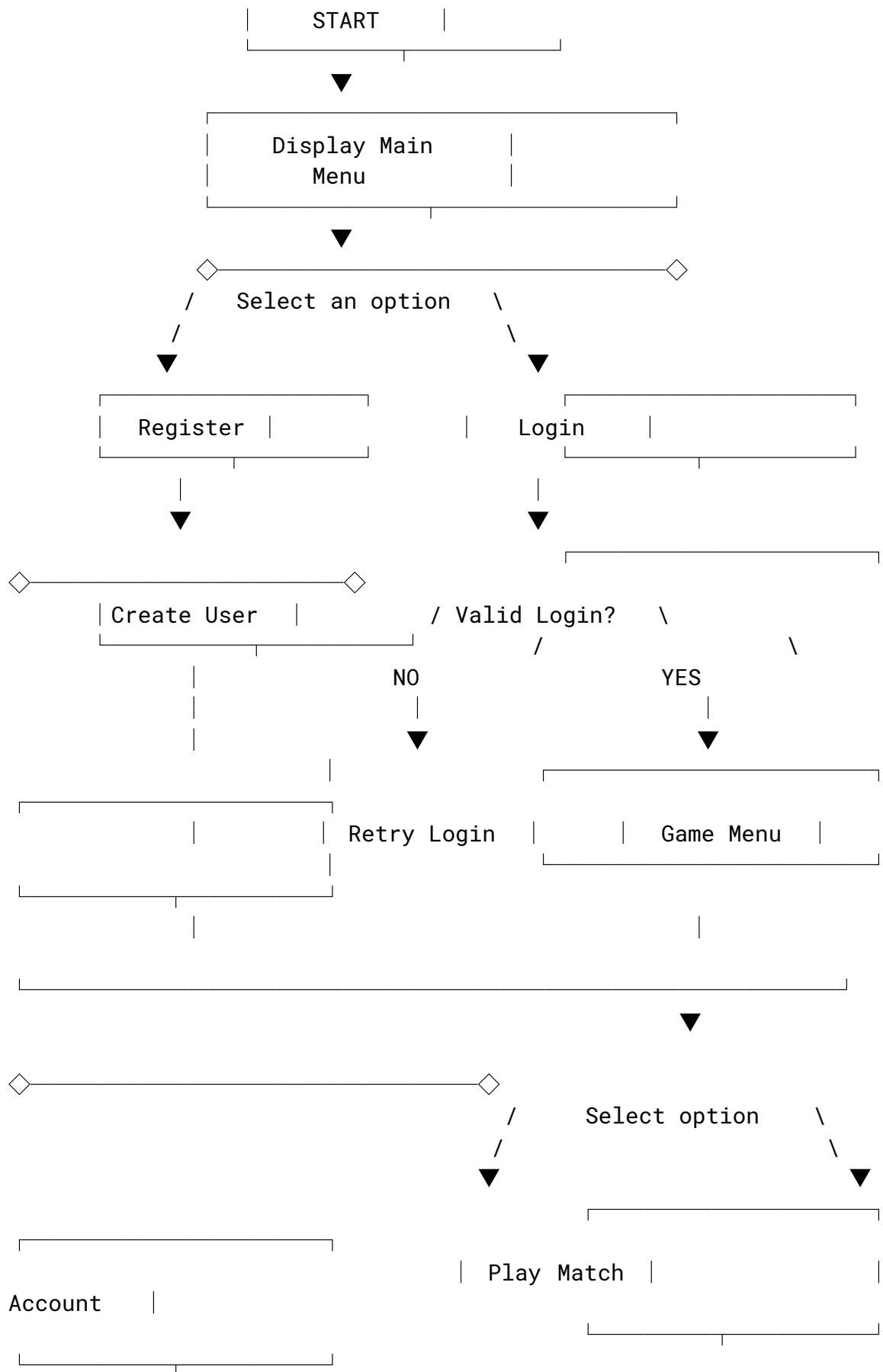
- Profile viewing
- Password modification
- Subscription management

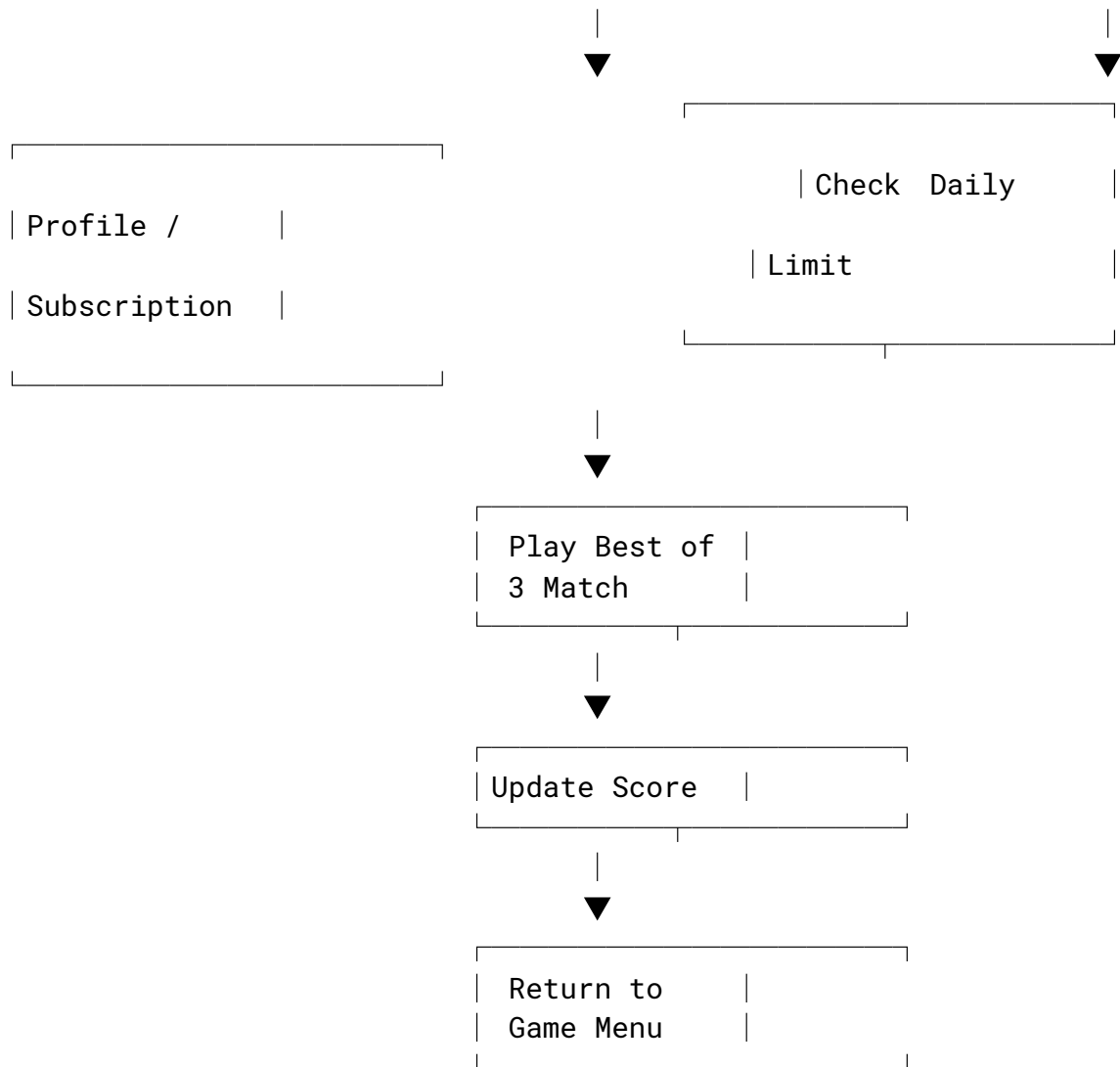
Storage Module

- Maintains user information using the users dictionary.
-

6.Process Flow/ Workflow Diagram

Main Application Flow





7. UML Diagrams

7.1 Use Case Diagram

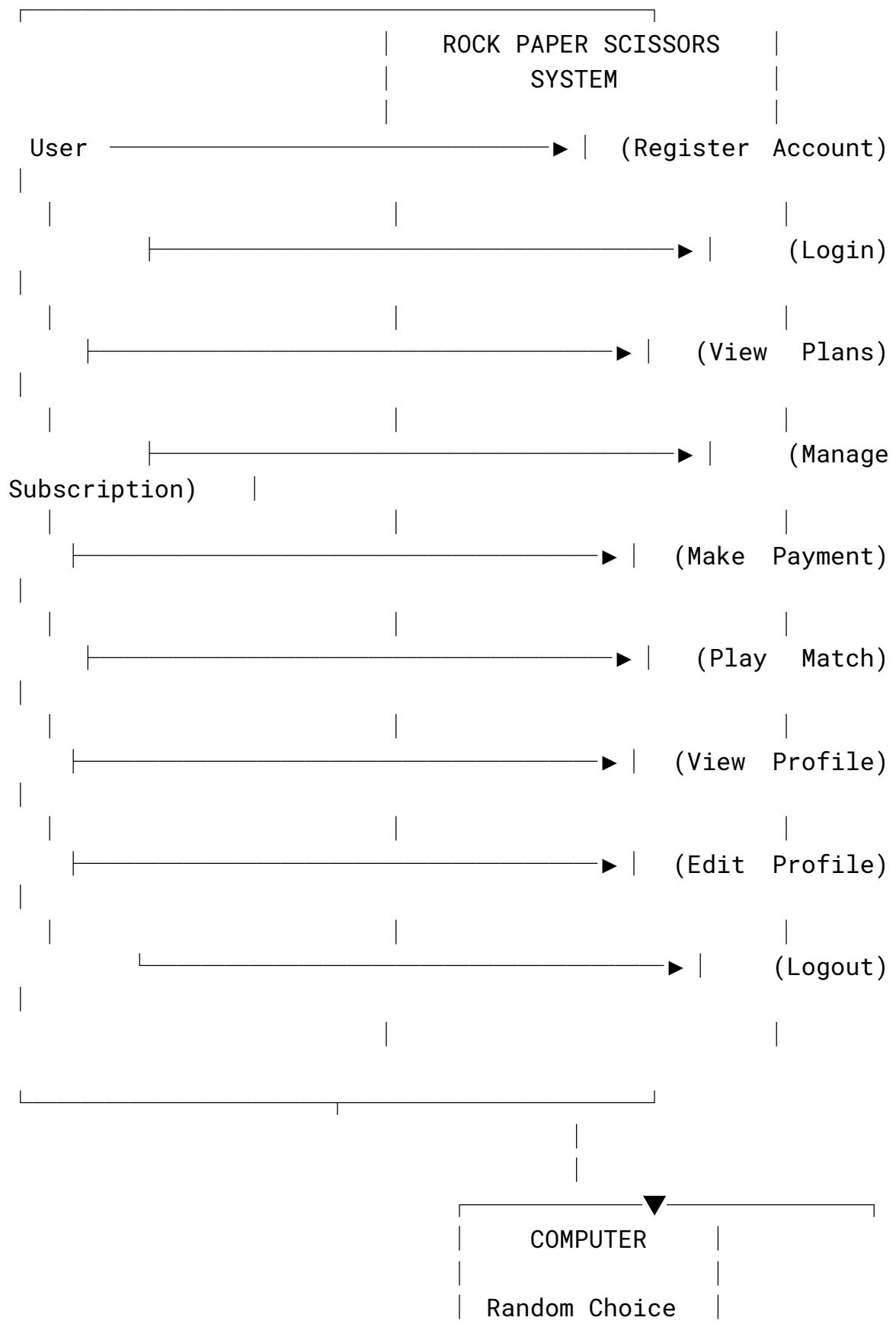
Actors

Primary Actor:

- User

External/Logical System Actor:

- Computer/Game Engine

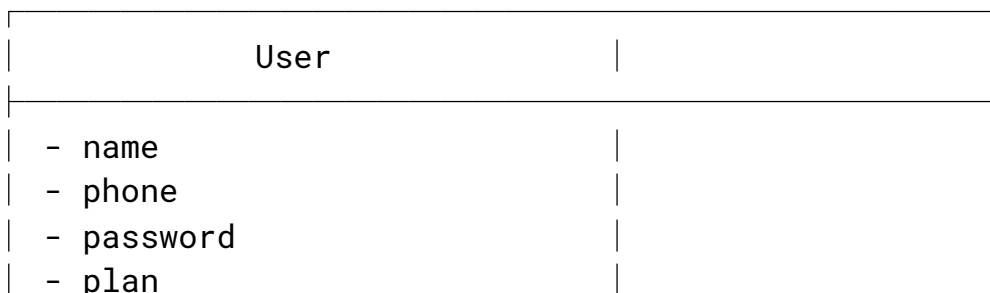


Use Cases

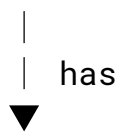
Use Case	Description
Register	Create a new user account
Login	Authenticate an existing user
Play Match	Play Rock Paper Scissors
View Rules	Display game instructions
View Profile	Display account information
Edit Profile	Change password
Manage Subscription	Select or change a plan
Make Payment	Simulate payment
Logout	End the current session

7.2 Class Diagram

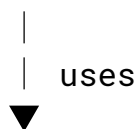
Although the program is primarily function-based rather than class-based, a UML class diagram can represent the logical components of the system.



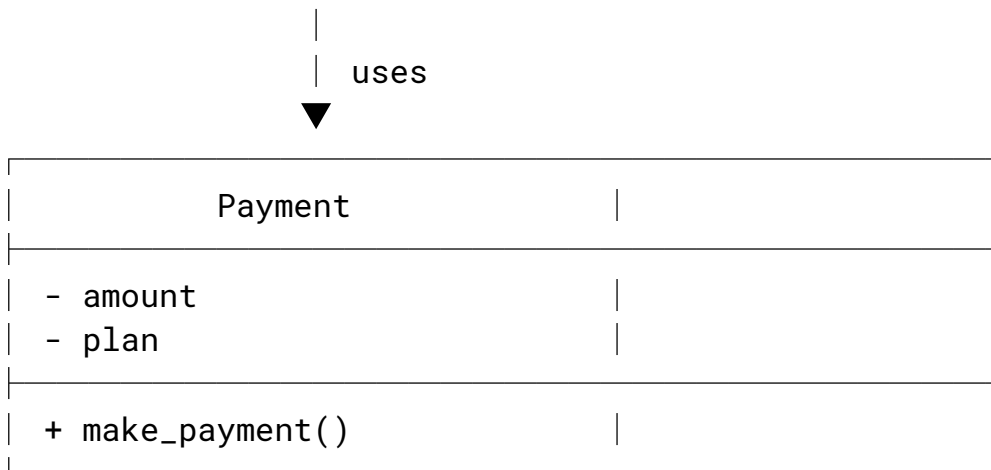
- matches_today	
- last_play_date	
- wins	
- losses	
- draws	
- payments	
+ register()	
+ login()	
+ edit_profile()	
+ view_profile()	



SubscriptionPlan	
- name	
- price	
- daily_limit	
+ show_plans()	
+ subscribe()	



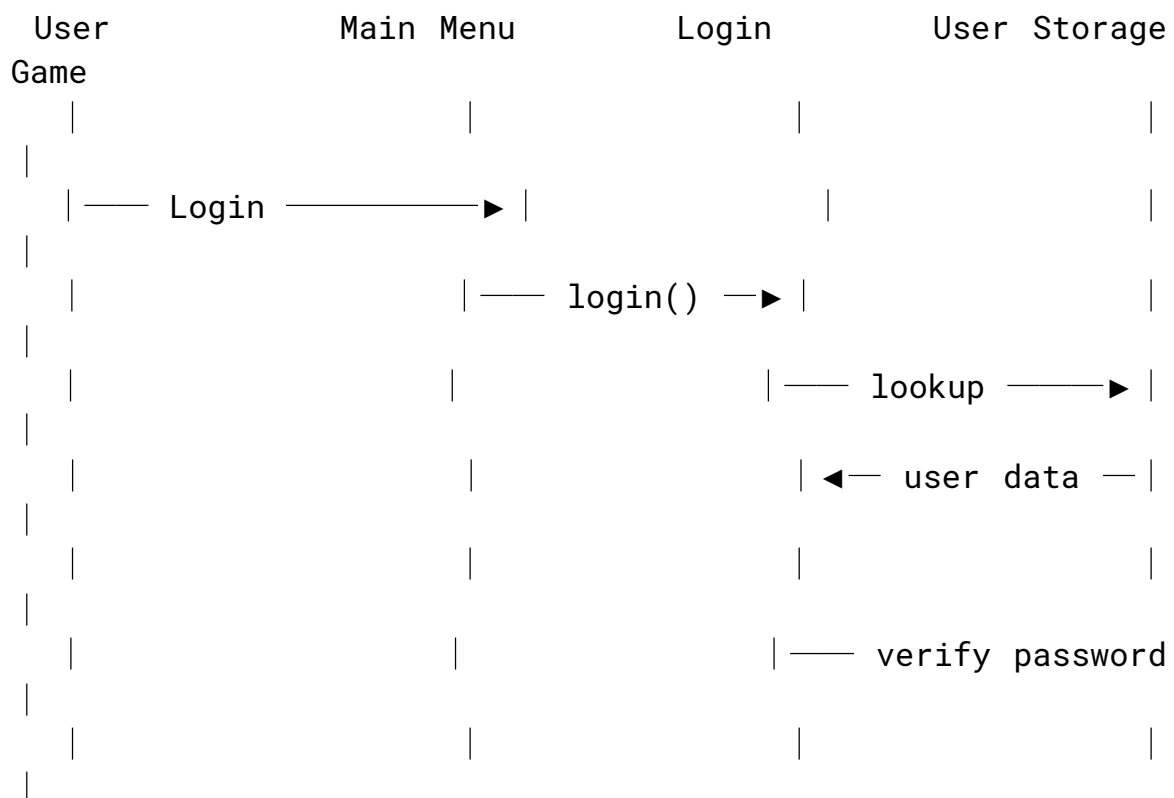
Game	
- player_score	
- computer_score	
- round_number	
+ get_player_choice()	
+ get_computer_choice()	
+ decide_winner()	
+ play_game()	

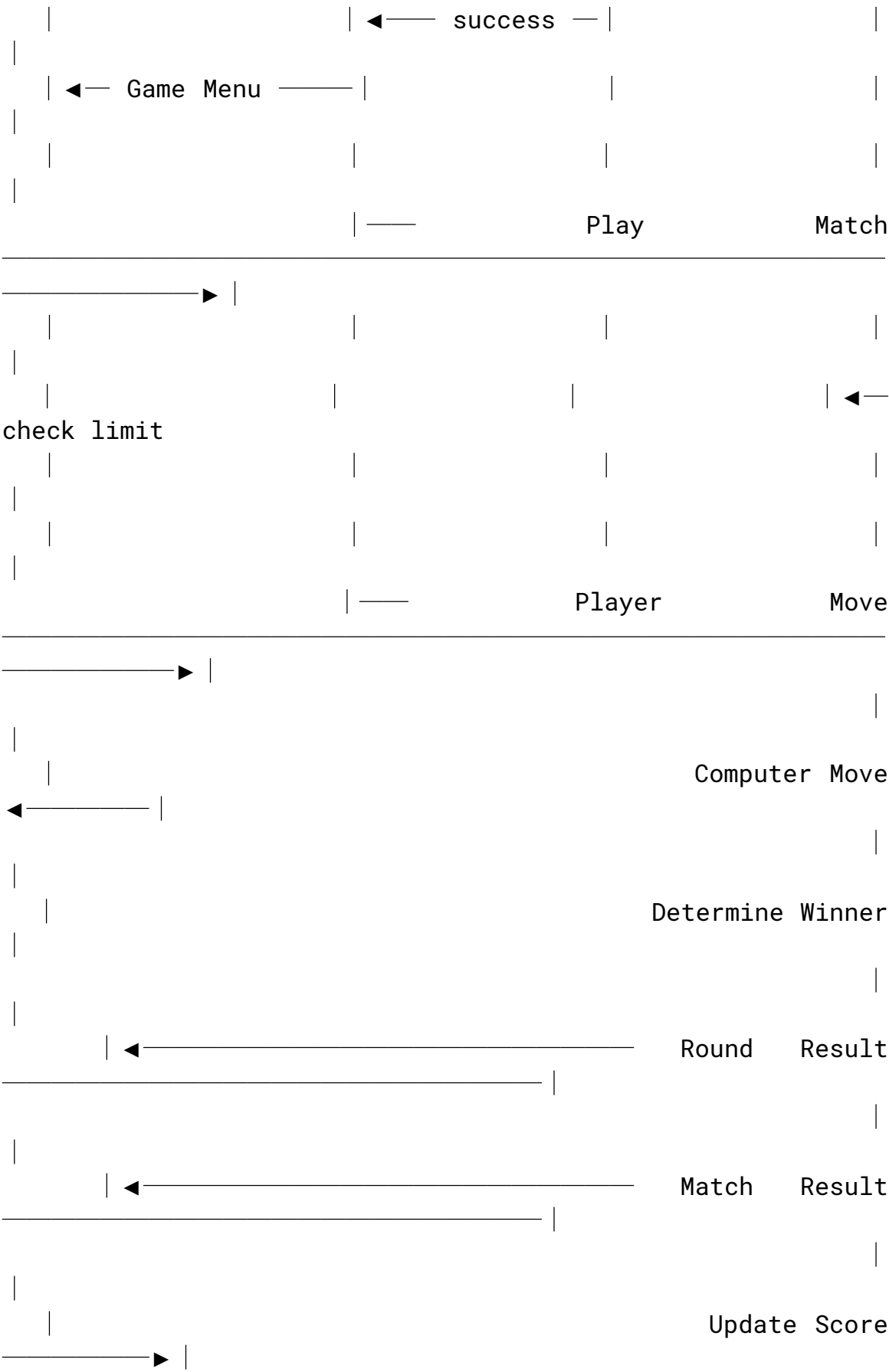


Note: This is a conceptual UML representation because the supplied implementation does not actually define Python classes.

7.3 Sequence Diagram

Login and Play Match





8. Database / Storage Design

Applicability

A traditional database is not used in the current project.

The program uses:

```
users = {}
```

This dictionary acts as temporary storage during execution.

Each phone number is used as a key:

```
users
├── Phone Number
│   └── User Information
└── ...
```

8.1 Storage Schema Design

The logical user record is:

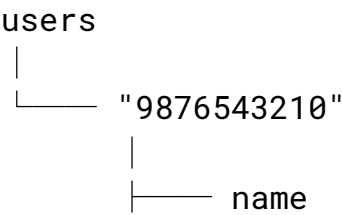
Field	Data Type	Description
-------	-----------	-------------

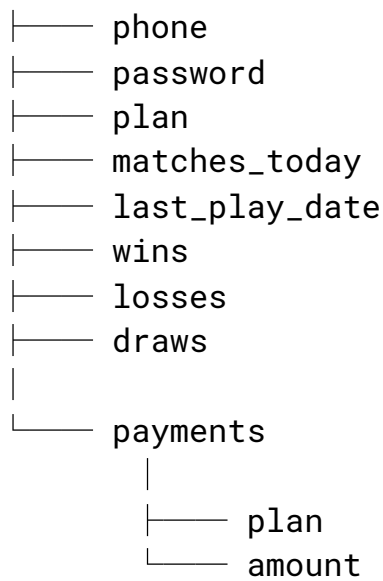
name	String	User's full name
phone	String	10-digit phone number
password	String	SHA-256 hashed password
plan	String	Free/Premium/Pro
matches_today	Integer	Matches played today
last_play_date	String	Last date played
wins	Integer	Total matches won
losses	Integer	Total matches lost
draws	Integer	Draw count
payments	List	Payment records

Payment Record

Field	Data Type	Description
plan	String	Purchased subscription
amount	Integer	Payment amount

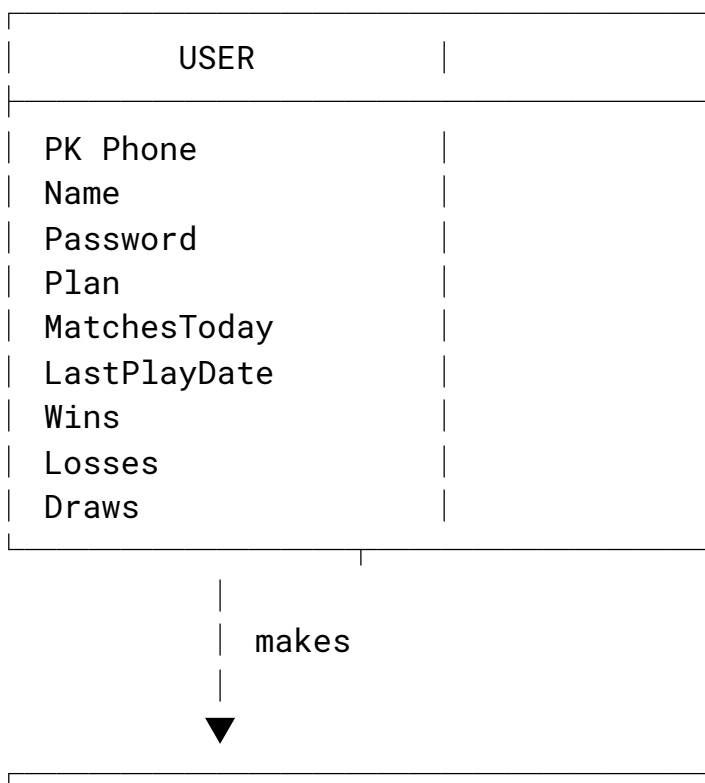
Example logical structure:





8.2 ER Diagram

Since there is no actual relational database, an ER diagram is optional. If your instructor requires one, you can represent the logical data relationships as follows:



PAYMENT	
Plan	
Amount	

9. Dataset / ML Requirements

Not applicable.

This project is a traditional rule-based Python application and does not use:

- Machine learning
- Artificial intelligence models
- Training datasets
- Classification/regression models
- Model evaluation

The computer's move is generated using Python's `random.choice()`:

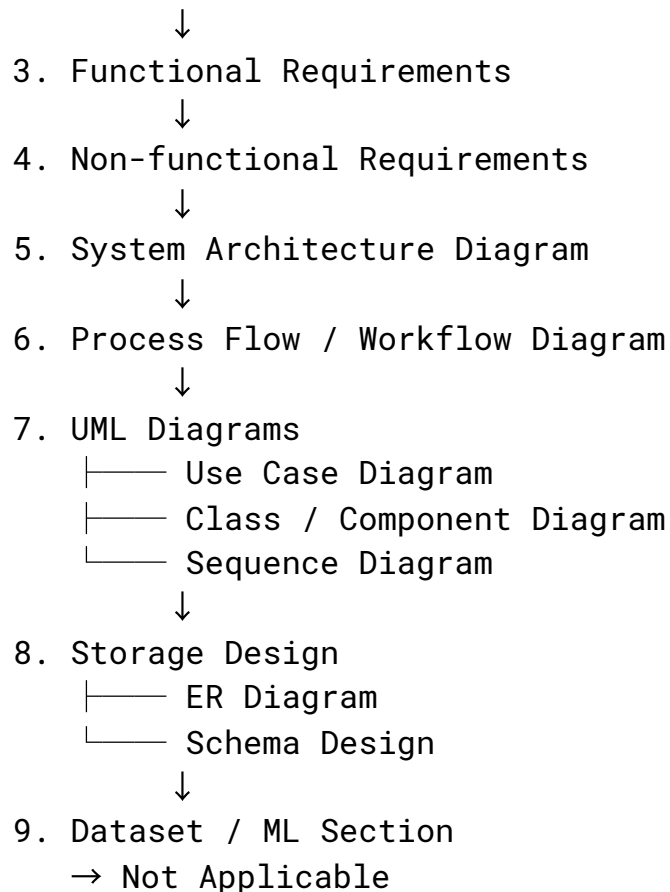
```
random.choice(CHOICES)
```

Therefore, dataset description, model selection rationale, and ML evaluation methodology are not required for this project.

10. Overall Project Structure

For your final report, the recommended order is:

1. Problem Statement
↓
2. Objectives



One important correction for your actual code: the draws field is displayed and initialized, but it is never incremented after a drawn match. Also, the Pro plan is implemented with a limit of 1000, not truly unlimited. These are worth correcting before submitting the project if your report claims otherwise.

Code -

|||||

ROCK PAPER SCISSORS

A simple console based Python game with:

- Registration (name, phone, password)**
- Login system (Phone number based)**
- Subscription plans (Free / Premium / Pro)**
- Payment option (simulated, offline)**
- Rock Paper Scissors game vs Computer**
- Basic score tracking**
- In-memory storage (Resets on exit)**

.....

```
import os
```

```
import re
```

```
import random
```

```
import hashlib
```

```
import time
```

CONSTANTS

CHOICES = ["rock", "paper", "scissors"]
SHORTCUTS = {"r": "rock", "p": "paper", "s": "scissors"}
PLANS = {
"1": {"name": "Free", "price": 0, "daily_limit": 5},
"2": {"name": "Premium", "price": 49, "daily_limit": 20},
"3": {"name": "Pro", "price": 99, "daily_limit": 1000},
}

BASIC HELPER FUNCTIONS

```
def clear_screen():
os.system("cls" if os.name == "nt" else "clear")

def pause():
input("\nPress Enter to continue...")

def print_line():
print("-" * 50)

def print_title():
print_line()
print("    ROCK - PAPER - SCISSORS  GAME")
print_line()

def hash_password(password):
return hashlib.sha256(password.encode()).hexdigest()

def is_valid_phone(phone):
return phone.isdigit() and len(phone) == 10

def is_valid_password(password):
if len(password) < 6:
return False

has_digit = any(ch.isdigit() for ch in password)
return has_digit
```

USER CREATION

```
def create_new_user(name, phone, password):  
    return {  
        "name": name,  
        "phone": phone,  
        "password": hash_password(password),  
        "plan": "Free",  
        "matches_today": 0,  
        "last_play_date": "",  
        "wins": 0,  
        "losses": 0,  
        "draws": 0,  
        "payments": [],  
    }
```

REGISTRATION AND LOGIN

```
def register(users):  
clear_screen()  
print_title()  
print("\n---- NEW USER REGISTRATION ----\n")  
  
phone = input("Enter your 10 digit phone number: ").strip()  
while not is_valid_phone(phone) or phone in users:  
    if phone in users:  
        print("An account with this phone number already  
exists.")  
    else:  
        print("Invalid phone number. Enter a 10 digit  
number.")  
    phone = input("Enter your 10 digit phone number:  
").strip()  
  
name = input("Enter your full name: ").strip()  
while len(name) < 2:  
    name = input("Please enter a valid name: ").strip()
```

```

password = input("Choose a password (min 6 chars, at least 1
number): ").strip()
while not is_valid_password(password):
    print("Password too weak. Try again.")
    password = input("Choose a password: ").strip()

confirm = input("Confirm password: ").strip()
while confirm != password:
    confirm = input("Passwords do not match. Confirm
password again: ").strip()

users[phone] = create_new_user(name, phone, password)

print("\nRegistration successful! You can now log in.")
pause()
return phone

```

def login(users):

clear_screen()

print_title()

print("\n---- LOGIN ----\n")

```

attempts = 0
while attempts < 3:
    phone = input("Phone Number: ").strip()
    password = input("Password: ").strip()

    user = users.get(phone)
    if user and user["password"] == hash_password(password):
        print(f"\nWelcome back, {user['name']}!")
        pause()
        return phone

    attempts += 1
    print(f"Incorrect phone number or password. Attempts
left: {3 - attempts}")

```

```
print("Too many failed attempts.")
pause()
return None
```

-

SUBSCRIPTION AND PAYMENT

-

```
def show_plans():
    print_line()
    print("SUBSCRIPTION PLANS")
    print_line()
    print("[1] Free    - Rs 0    - 5 matches per day")
    print("[2] Premium - Rs 49   - 20 matches per day")
    print("[3] Pro     - Rs 99   - Unlimited matches")
    print_line()
    def make_payment(amount):
        print(f"\nAmount to pay: Rs {amount}")
```

```

confirm = input("Proceed with payment? (y/n): ").strip().lower()
if confirm != "y":
    print("Payment cancelled.")
    return False

    card = input("Enter 16 digit card number: ").strip()
    while not (card.isdigit() and len(card) == 16):
        card = input("Invalid card number. Enter 16 digits: ").strip()

    expiry = input("Enter expiry date (MM/YY): ").strip()
    while not re.match(r"^\d{2}/\d{2}$", expiry):
        expiry = input("Invalid format. Enter expiry as MM/YY: ").strip()

    cvv = input("Enter CVV (3 digits): ").strip()
    while not (cvv.isdigit() and len(cvv) == 3):
        cvv = input("Invalid CVV. Enter 3 digits: ").strip()

    print("Processing payment", end="")
    for _ in range(3):
        time.sleep(0.3)
        print(".", end="", flush=True)
    print("\nPayment successful!")
    return True

def subscription_menu(users, phone):
    user = users[phone]
    while True:
        clear_screen()
        print_title()
        print(f"\nCurrent Plan: {user['plan']}")
        show_plans()
        print("[0] Back to main menu")

```


ROCK PAPER SCISSORS GAME LOGIC

—

```
def get_computer_choice():
    return random.choice(CHOICES)

def get_player_choice():
    while True:
        move = input("Choose (r)ock, (p)aper, or (s)cissors: ")
        move = move.strip().lower()

        if move in CHOICES:
            return move

        if move in SHORTCUTS:
            return SHORTCUTS[move]

        print("Invalid input. Try again.")

def decide_winner(player, computer):
    if player == computer:
        return "draw"

    wins_against = {"rock": "scissors", "scissors": "paper", "paper": "rock"}

    if wins_against[player] == computer:
        return "player"

    return "computer"
```

```
def check_daily_limit(users, user):  
today = time.strftime("%Y-%m-%d")  
if user["last_play_date"] != today:  
user["last_play_date"] = today  
user["matches_today"] = 0  
  
    limit = PLANS["1"]["daily_limit"]  
    for plan in PLANS.values():  
        if plan["name"] == user["plan"]:  
            limit = plan["daily_limit"]  
    return user["matches_today"] < limit
```

```
def play_game(users, phone):
```

```
user = users[phone]
```

```
    if not check_daily_limit(users, user):  
        print(f"\nYou have reached your daily match limit for  
the {user['plan']} plan.")  
        print("Upgrade your subscription to play more matches  
today.")  
        pause()  
        return
```

```
    print("\nBest of 3 rounds. First to win 2 rounds wins the  
match!\n")
```

```
    player_score = 0  
    computer_score = 0  
    round_number = 0
```

```
    while player_score < 2 and computer_score < 2:  
        round_number += 1  
        print_line()  
        print(f"Round {round_number}")  
  
        player_move = get_player_choice()  
        computer_move = get_computer_choice()
```

```

    print(f"You chose: {player_move} | Computer chose:
{computer_move}")

    result = decide_winner(player_move, computer_move)
    if result == "player":
        print("You win this round!")
        player_score += 1
    elif result == "computer":
        print("Computer wins this round!")
        computer_score += 1
    else:
        print("This round is a draw!")

print_line()
if player_score > computer_score:
    print(f"YOU WON THE MATCH! {player_score} -
{computer_score}")
    user["wins"] += 1
else:
    print(f"COMPUTER WON THE MATCH. {player_score} -
{computer_score}")
    user["losses"] += 1

user["matches_today"] += 1
pause()

```

def show_rules():

clear_screen()

print_title()

print("""

HOW TO PLAY

Rock beats Scissors

Scissors beats Paper

Paper beats Rock

Each match is Best of 3 rounds.

Free plan: 5 matches/day. Premium: 20/day. Pro: unlimited.

""")

pause()

ACCOUNT MENU

def view_profile(user):

clear_screen()

print_title()

print("\nMY PROFILE\n")

print(f"Name : {user['name']}")

print(f"Phone : {user['phone']}")

print(f"Plan : {user['plan']}")

**print(f"Wins/Losses/Draws:
{user['wins']}/{user['losses']}/{user['draws']}")**

pause()

def edit_profile(users, phone):

```
user = users[phone]  
clear_screen()  
print_title()  
print("\nEDIT PROFILE (leave blank to keep current value)\n")
```

```
change = input("Change password? (y/n): ").strip().lower()  
if change == "y":  
    new_password = input("New password: ").strip()  
    if is_valid_password(new_password):  
        user["password"] = hash_password(new_password)  
        print("Password updated.")  
    else:  
        print("Password too weak. Not changed.")  
  
print("\nProfile updated.")  
pause()
```

```
def account_menu(users, phone):
```

```
while True:
```

```
user = users[phone]
```

```
clear_screen()
```

```
print_title()
```

```
print("\nACCOUNT MENU\n")
```

```
print("[1] View Profile")
```

```
print("[2] Edit Profile")
```

```
print("[3] Manage Subscription")
```

```
print("[0] Back to main menu")
```

```
choice = input("\nEnter choice: ").strip()  
if choice == "1":  
    view_profile(user)  
elif choice == "2":  
    edit_profile(users, phone)  
elif choice == "3":
```

```
        subscription_menu(users, phone)
    elif choice == "0":
        break
    else:
        print("Invalid choice.")
        pause()
```

GAME MENU (after login)

```
def game_menu(users, phone):
while True:
    user = users[phone]
    clear_screen()
    print_title()
    print(f"\nLogged in as: {user['name']} ({phone}) | Plan:
{user['plan']}\n")
    print("[1] Play a Match")
    print("[2] How to Play")
    print("[3] Account & Subscription")
    print("[0] Logout")

    choice = input("\nEnter choice: ").strip()
    if choice == "1":
```

```
        play_game(users, phone)
    elif choice == "2":
        show_rules()
    elif choice == "3":
        account_menu(users, phone)
    elif choice == "0":
        print("Logging out...")
        time.sleep(1)
        break
    else:
        print("Invalid choice.")
        pause()
```

WHY SIGN IN? (info screen)

```
def why_signin():  
clear_screen()
```

```
print_title()
```

```
print("""
```

WHY DO GAMES ASK YOU TO SIGN IN BEFORE PLAYING?

1. **Saving progress** - your score and stats stay linked to your account.
2. **Personalisation** - the game remembers your name and settings.
3. **Fair usage** - daily play limits stop misuse of free features.
4. **Subscriptions** - paid plans need an account to track who has paid.
5. **Security** - passwords are hashed, not stored as plain text.
6. """)
7. **pause()**

-

MAIN MENU

-

```
def main():
```

```
users = {} # In-memory dictionary; data will be lost when the
```

script closes.

```
while True:
    clear_screen()
    print_title()
    print("\n[1] Login")
    print("[2] Register")
    print("[3] Why sign in? (Info)")
    print("[4] View Subscription Plans")
    print("[0] Exit")

    choice = input("\nEnter choice: ").strip()

    if choice == "1":
        phone = login(users)
        if phone:
            game_menu(users, phone)
    elif choice == "2":
        phone = register(users)
        if phone:
            game_menu(users, phone)
    elif choice == "3":
        why_signin()
    elif choice == "4":
        clear_screen()
        print_title()
        show_plans()
        pause()
    elif choice == "0":
        print("Thanks for playing! Goodbye.")
        break
    else:
        print("Invalid choice.")
        pause()
```

if name == "main":

main()

Outputs

```
Problems Output Debug Console Terminal Ports

nishthajain@Mac / % /usr/local/bin/python3 /Users/nishthajain/Untitled-2.py
ROCK - PAPER - SCISSORS GAME
-----

[1] Login
[2] Register
[3] Why sign in? (Info)
[4] View Subscription Plans
[0] Exit

Enter choice: █
```

```
Problems Output Debug Console Terminal Ports

ROCK - PAPER - SCISSORS GAME
-----

--- NEW USER REGISTRATION ---

Enter your 10 digit phone number: 9630772080
Enter your full name: nishtha
Choose a password (min 6 chars, at least 1 number): nisht1
Confirm password: nisht1

Registration successful! You can now log in.

Press Enter to continue...█
```

```
Problems Output Debug Console Terminal Ports

Logged in as: nishtha (9630772080) | Plan: Free

[1] Play a Match
[2] How to Play
[3] Account & Subscription
[0] Logout

Enter choice: 1

Best of 3 rounds. First to win 2 rounds wins the match!

-----

Round 1
Choose (r)ock, (p)aper, or (s)cissors: █
```

```
Problems Output Debug Console Terminal Ports

[0] Logout

Enter choice: 1

Best of 3 rounds. First to win 2 rounds wins the match!

-----

Round 1
Choose (r)ock, (p)aper, or (s)cissors: r
You chose: rock | Computer chose: rock
This round is a draw!

-----

Round 2
Choose (r)ock, (p)aper, or (s)cissors: █
```

```
Problems Output Debug Console Terminal Ports

Enter choice: 1

Best of 3 rounds. First to win 2 rounds wins the match!

-----

Round 1
Choose (r)ock, (p)aper, or (s)cissors: r
You chose: rock | Computer chose: rock
This round is a draw!

-----

Round 2
Choose (r)ock, (p)aper, or (s)cissors:
Invalid input. Try again.
Choose (r)ock, (p)aper, or (s)cissors: s█
```

```
Problems Output Debug Console Terminal Ports

ROCK - PAPER - SCISSORS GAME
=====

HOW TO PLAY
Rock beats Scissors
Scissors beats Paper
Paper beats Rock

Each match is Best of 3 rounds.
Free plan: 5 matches/day. Premium: 20/day. Pro: unlimited.

Press Enter to continue...|
```

```
Problems Output Debug Console Terminal Ports

ROCK - PAPER - SCISSORS GAME
=====

ACCOUNT MENU
[1] View Profile
[2] Edit Profile
[3] Manage Subscription
[0] Back to main menu

Enter choice: |
```

```
Problems Output Debug Console Terminal Ports

ROCK - PAPER - SCISSORS GAME
=====

Current Plan: Free
=====
SUBSCRIPTION PLANS
=====
[1] Free    - Rs 0    - 5 matches per day
[2] Premium - Rs 49   - 20 matches per day
[3] Pro     - Rs 99   - Unlimited matches
=====
[0] Back to main menu

Choose a plan number to subscribe, or 0 to go back: |
```

```
Problems Output Debug Console Terminal Ports

Current Plan: Free
=====
SUBSCRIPTION PLANS
=====
[1] Free    - Rs 0    - 5 matches per day
[2] Premium - Rs 49   - 20 matches per day
[3] Pro     - Rs 99   - Unlimited matches
=====
[0] Back to main menu

Choose a plan number to subscribe, or 0 to go back: 2

Amount to pay: Rs 49
Proceed with payment? (y/n): |
```

```
Problems Output Debug Console Terminal Ports

[1] Free    - Rs 0    - 5 matches per day
[2] Premium - Rs 49   - 20 matches per day
[3] Pro     - Rs 99   - Unlimited matches
=====
[0] Back to main menu

Choose a plan number to subscribe, or 0 to go back: 2

Amount to pay: Rs 49
Proceed with payment? (y/n): y
Enter 16 digit card number: 6616171815233452
Enter expiry date (MM/YY): 66/78
Enter CVV (3 digits): 667|
```

